

Artificial Intelligence and Knowledge Engineering

Assignment 1

Michał Karol, Paweł Narolski, Piotr Syga

February 27th, 2026

1 Exercise goal

This assignment aims to develop practical skills in solving optimization problems and implementing the algorithmic techniques covered in lectures. Upon completion, you should:

1. Understand the fundamental concepts of optimization,
2. Recognize the computational challenges inherent in finding exact solutions, and
3. Learn to apply practical approaches when computational resources are limited.

In particular, you will be able to understand how and when the approximation algorithms and heuristics are used. Furthermore, you will become familiar with various heuristic search techniques, and understand their application to pathfinding problems.

2 Theoretical introduction

In this section you will find information useful during solving the stated problems. It is assumed, that after finishing the exercise students are fully knowledgeable about theoretical concepts described below.

2.1 Definitions

Definition 1 (Optimization problem). *Optimization problem is a set of constraints (given in the form of inequalities or equalities with the decision variables) with the objective function. Let S be the feasible set in the assumed problem, i.e., the set of such S that satisfy all constraints, and $f(\bar{x}) : \mathbb{K}^n \rightarrow \mathbb{K}$ function to evaluate the quality of the solution. We define the minimization problem finding such a $s^* \in S$ for which $s^* = \min_s f(s)$ and $f(s)$ is a cost (or loss) function. Alternatively, a problem is called maximizing if the goal is to find such a $s^* \in S$ for which every $s^* = \max_{s \in S} f(s)$, where $f(s)$ is a utility function.*

One may easily note that every minimization problem could be transformed into maximizing problem.

Definition 2 (Optimal solution). *Let S be a feasible set for the (minimization) problem $\mathcal{P}$ with cost function f . Solution $s^* \in S$ is a global optimum iff $\forall_{s \in S} f(s^*) \leq f(s)$.*

Definition 3 (Approximation). *Let $\text{Opt}(x) = \min_{s \in S(x)} f(s)$. For algorithm M solving optimization problem ($M(x) \in S(x)$), we call it ε -approximation for $\varepsilon \geq 0$, if and only if for each x we have*

$$\frac{|f(M(x)) - \text{Opt}(x)|}{\max\{\text{Opt}(x), f(M(x))\}} \leq \varepsilon.$$

Definition 4 (Heuristic). *Let P be an optimization problem, and S a set of solutions for problem P . Let $f : S \rightarrow \mathbb{R}$ be a cost function. Let s^* be a globally optimal solution.*

Heuristic H for problem P is a function $H : S \rightarrow S$ returning solution $s' \in S$ for problem P , such that for most cases solution s is good enough, meaning that difference $|s^ - s'|$ is not significant.*

Heuristics could use simplifications, approximations, and empirical rules in order to reduce computational complexity or in order to increase the chances of finding a good solution in a relatively short time.

A formal definition of heuristic implies that heuristics are not exact algorithms, and are only methods of searching for a good approximate solution to the problem. Heuristics could be used on their own or in conjunction with exact algorithms to improve the performance and quality of found solutions. Heuristics could be used for non-approximable problems.

Heuristics are applied in order to solve the traveling salesman problem (TSP), where the target is to find the shortest closed path connecting all cities out of the set of V .

Definition 5 (Traveling salesman problem (TSP)). *Let $G = (V, E)$ be a graph, where V is a set of nodes (cities to be visited) with the size of n , and E is a set of edges connecting a pair of cities. Each edge $(u, v) \in E$ has non-negative cost $d(u, v)$ representing distance between cities u and v .*

The goal is to find the shortest closed path that visits each city exactly once and then returns back to the starting point. Formally, one has to find a permutation $\pi = (v_1, v_2, \dots, v_n)$ of set V , such that sum of edges costs $D = d((\pi_1, \pi_2)) + d((\pi_2, \pi_3)) + \dots + d((\pi_{n-1}, \pi_n)) + d((\pi_n, \pi_1))$ is minimal.

2.2 A* search algorithm

A* algorithm is a heuristic algorithm applied to search for the shortest path on a graph. The algorithm is an extension of the Dijkstra algorithm, using an additional approximation estimation module from the node to the target. Note that, with heuristics that are not properly designed, the algorithm can return results far from the optimum. Algorithm was proposed in 1968 by Peter E. Hart, Nils John Nilsson and Bertram Raphael.

A* search algorithm is based on the minimization of a cost function defined as $f(curr, next) = g(curr, next) + h(next, end)$, where $g(curr, next)$ is a cost function of going to the node $next$ and $h(next, end)$ estimates cost from $next$ node to the end node.

Commonly used cost estimations:

- Manhattan distance: $h(curr, next) = |curr.x - next.x| + |curr.y - next.y|$
- Euclidean distance: $h(curr, next) = \sqrt{(curr.x - next.x)^2 + (curr.y - next.y)^2}$

The main idea behind the A* algorithm is using of two lists: open - consisting of nodes that were visited, but not all neighbors were visited, and closed list where all nodes on the list were visited as well as all the neighbors.

Algorithm 1 A* algorithm

Input: start, end

Output: List of nodes and edges creating the found path

```
1:  $start.g \leftarrow 0$ 
2:  $start.h \leftarrow 0$ 
3:  $start.f \leftarrow start.g + start.h$ 
4:  $open \leftarrow list([start])$ 
5:  $closed \leftarrow list()$ 
6: while  $len(open) > 0$  do
7:    $node \leftarrow null$ 
8:    $node\_cost \leftarrow +Inf$ 
9:   for  $test\_node$  in  $open$  do
10:    if  $f(test\_node) < node\_cost$  then
11:       $node \leftarrow test\_node$ 
12:       $node\_cost \leftarrow f(test\_node)$ 
13:   if  $node == end$  then
14:     Solution found
15:    $open \leftarrow (open - node)$ 
16:    $closed \leftarrow (closed + node)$ 
17:   for  $next\_node$  in  $neighbourhood(node)$  do
18:    if  $next\_node$  not in  $open$  and  $next\_node$  not in  $closed$  then
19:       $open \leftarrow (open + next\_node)$ 
20:       $next\_node.h = h(next\_node, end)$ 
21:       $next\_node.g = node.g + g(node, next\_node)$ 
22:       $next\_node.f = next\_node.g + next\_node.h$ 
23:   else
24:    if  $next\_node.g > node.g + g(node, next\_node)$  then
25:       $next\_node.g = node.g + g(node, next\_node)$ 
26:       $next\_node.f = next\_node.g + next\_node.h$ 
27:    if  $next\_node$  in  $closed$  then
28:       $otarte \leftarrow (open + next\_node)$ 
29:       $closed \leftarrow (closed - next\_node)$ 
```

Dijkstra's algorithm Dijkstra's algorithm is a shortest-path search algorithm for a non-negative weighted graph with a single source. The algorithm starts with a set of shortest distances from the source to the nodes and updates it while visiting nodes.

Let $G = (V, E)$ be a weighted graph with a source s , set of nodes V , and set of edges E . Let $w : E \rightarrow \mathbb{R}$ be the cost function for traveling the edge. For each $v \in V$, let $d(v)$ be the cost of the shortest path from s to v , and $p(v)$ be a function returning the preceding node of v for the currently shortest path from s to v .

Dijkstra's algorithm works in the following way:

1. Initialize $d(s) = 0$ and $d(v) = \infty$ for each $v \in V \setminus \{s\}$.
2. Create set Q containing all nodes of graph G .
3. While Q is not empty, do:
 - (a) Select node $u \in Q$ with smallest $d(u)$ and remove it from Q
 - (b) For each v such that $\exists_{(u,v) \in E} d(v) > d(u) + w(u, v)$ update $d(v) = d(u) + w(u, v)$ and $p(v) = u$
4. Return d and p

Set Q represents the set of nodes, that are not yet visited, hence with the distance from s not yet updated. In step 3a, the node from Q with smallest value $d(u)$ is selected, and added to the set of nodes with shortest

distance from the source. Furthermore, the shortest distances for all neighbors of the node u are updated, if $d(u) + w(u, v)$ is smaller than already stored $d(v)$. This way algorithm finds the shortest path from s to every node in a graph.

2.3 Local search and Tabu Search

Local search The local search algorithm is a metaheuristic that involves an iterative search in the neighborhood of the current solution, with the intention of finding a better solution.

Let P denote the optimization problem and s be the current solution. Let $N(s)$ be the neighborhood of s , that is, the set of solutions that can be achieved by changing a fixed number of parameters (e.g., a different evaluation of one of the variables) of the current solution s . Let $f(s)$ denote the value of the objective function for the solution of s .

The algorithm starts from generating the initial solution s_0 . Then, at each iteration, the algorithm generates a new solution $s'(s)$. If $f(s') < f(s)$, the algorithm accepts the new solution s' and sets it as the current solution s . Otherwise, it rejects the new solution and continues searching the neighborhood. The search ends when the stop criterion, which is one of the parameters of the heuristics (e.g., the maximum number of iterations, the maximum number of iterations without updating s), is reached. The local search algorithm is sensitive to getting stuck in the local optimum of the objective function.

Tabu search Tabu search algorithm is a metaheuristic that involves iteratively searching the neighborhood of the current solution, taking into account the set of moves that are not allowed (tabu). Compared to local search, it requires additional memory but eliminates the retesting of the same (or similar) solutions to get out of the local optimum.

The main steps of Tabu search:

1. We generate the initial solution s_0 .
2. We set the current solution s as s_0 , the tabu set as $T = \emptyset$ and the best known solution as $s^* = s$.
3. We define the neighborhood $N(s)$.
4. Until we meet the stop condition:
 - (a) We generate the neighborhood of the current solution $N(s)$.
 - (b) We test the whole neighborhood or (deterministically) sample $N(s)$.
 - (c) We select the best of the neighbors, such that $s_i \in N(s) \setminus T$.
 - (d) $T = T \cup N(s)$
 - (e) We set $s = s_i$, if $f(s) \leq f(s^*)$, then we set $s^* = s$.
5. We return the solution s^* .

It can be noted that depending on the presented optimization problem P , the Tabu search algorithm should be adapted to the specific conditions of the task. An example is the time-consuming nature of searching $N(s)$, which, for example, for points in a sphere of radius r is infinite. In such a case, a sampling (e.g., deterministic or Monte Carlo method) of solutions lying in the neighborhood of s is performed. In addition, it is easy to see that the array of forbidden solutions T increases with the number of steps, requiring more and more memory. The basic method of solving this problem is to set the size of the array $t = T \setminus |T|$ and treat it as a FIFO queue, where in case of overflow the oldest entry is removed. The choice of t should depend on the characteristics of $N(s)$, since t that is too small for large neighborhoods makes it easy to create cycles of length greater than t , causing the same solutions to be revisited, thus problems similar to those of local search. On the other hand, too large t for small neighborhoods results in the consumption of redundant memory and the failure to remove solutions from T . In order to admit solutions from T that, due to sampling, ended up in the array of forbidden solutions without checking the value of the utility function, the aspiration function (or array) A can be used, which specifies that if a solution is good enough, it is considered as a possible solution despite its presence in T .

It is worth noting that we do not always have to keep the list of complete solutions in T . Using the specifics of the problem (or representing the set of solutions as an n -dimensional cube), we can modify T :

- we can prohibit the vertices – instead of prohibiting the entire solution, we prohibit a fixed value at one of the coordinates, e.g., in TSP we prohibit all solutions that have a city j at position i or prohibit changing the position of a city i .
- we prohibit the edges – this approach prohibits movements inside a fixed plane (two or more coordinates), e.g., for TSP in 2-opt prohibits the removal of one of the edges.
- we prohibit the edges with one of the vertices – we forbid moving along a fixed straight line (for two coordinates) or inside a fixed plane (more coordinates), e.g., for TSP, all edges that leave a fixed vertex i are forbidden.

Below, an example of a pseudocode that uses Tabu search for solving traveling salesman problem is shown.

Algorithm 2 Knox algorithm: Tabu search for TSP

```

1:  $k \leftarrow 0$ 
2: randomly select the starting solution  $s$ 
3:  $s^* \leftarrow s$ 
4:  $T \leftarrow \emptyset$ 
5: while  $k < \text{STEP\_LIMIT}$  do
6:    $i \leftarrow 0$ 
7:   while  $i < \text{OP\_LIMIT}$  do
8:     define  $N(s)$ ,  $A$ 
9:     select the best  $s' \in (N(s) \setminus T) \cup A$ 
10:    update  $T$ 
11:    if  $D(s') < D(s)$  then
12:      update the local optimum  $s \leftarrow s'$ 
13:     $i++$ 
14:   $k++$ 
15:  if  $D(s) < D(s^*)$  then
16:     $s^* \leftarrow s$ 

```

Knox's algorithm allows us to consider a neighboring solution that is in the Tabu list if it is good enough. In doing so, it uses an Aspiration (A) array.

An example of an Aspiration array could be implemented by remembering the history of k last steps of an algorithm. In such a case, in addition to the table T , let's store a history H describing the number of modifications to the variable in h last steps. Let's set $0 < \text{varepsilon} < 1$ and $A_i = f(s_i) + \text{varepsilon}(k - H(i))$. If $f(s) > A_i$, we select the neighbor s_i .

3 Materials

The timetable data is provided in the **GTFS** (General Transit Feed Specification) format, published by Koleje Dolnośląskie S.A., valid from March 3rd, 2026 to December 12th, 2026. The dataset is distributed as a set of comma-separated text files (`.txt`), each representing a different aspect of the transit network. Together, these files fully describe the topology, scheduling, and service patterns of the Koleje Dolnośląskie railway network.

Below is a detailed description of each file and its fields. For a comprehensive specification of the GTFS format, refer to the official documentation at <https://gtfs.org/schedule/reference/>. Note that, upon implementing the usage of the GTFS format, you will be able to use data from other transit agencies throughout the world.

3.1 agency.txt – Transit agency information

Contains metadata about the transit operator. In this dataset, there is a single agency: Koleje Dolnośląskie (agency ID: 279).

- `agency_id` – unique identifier of the transit agency
- `agency_name` – full name of the agency
- `agency_url` – URL of the agency's website
- `agency_timezone` – timezone in which the agency operates (e.g., Europe/Warsaw)
- `agency_phone` – contact phone number
- `agency_email` – contact email address
- `agency_fare_url` – URL for purchasing tickets

3.2 `routes.txt` – Route definitions (57 routes)

Defines the transit routes (lines) operated by the agency. Each route corresponds to a named rail line (e.g., D1, D10, D66). A single route may have multiple trips running at different times and on different days.

- `route_id` – unique identifier of the route
- `agency_id` – reference to the operating agency (foreign key to `agency.txt`)
- `route_short_name` – short public-facing name of the route (e.g., D1, D66)
- `route_long_name` – longer descriptive name of the route (used when `route_short_name` is empty, e.g., KD Premium Nadmorski)
- `route_type` – type of transport; value 2 denotes rail
- `route_color` – hexadecimal color code for route display (e.g., F6C32F)
- `route_text_color` – hexadecimal text color for labels on the route color background

3.3 `stops.txt` – Stop locations (1 108 entries)

Describes all stops (stations and platforms) in the network. Each stop has geographic coordinates in WGS84 and may represent either a parent station (`location_type` = 1) or a specific platform within a station (`location_type` = 0). Platform entries reference their parent station via the `parent_station` field.

- `stop_id` – unique identifier of the stop or platform
- `stop_code` – public-facing code of the stop (may be empty for platforms)
- `stop_name` – human-readable name of the stop (e.g., Wrocław Główny, Jelenia Góra)
- `stop_desc` – optional textual description of the stop
- `stop_lat` – latitude of the stop in WGS84 coordinate system
- `stop_lon` – longitude of the stop in WGS84 coordinate system
- `location_type` – 0 = platform/stop, 1 = parent station
- `parent_station` – `stop_id` of the parent station (empty for parent stations themselves)
- `platform_code` – platform designation within the station (e.g., II, 3)

3.4 trips.txt – Trip definitions (3 260 trips)

Defines individual trips, i.e., concrete runs of a vehicle along a route at a specific time. Each trip belongs to exactly one route and one service pattern (determining the days on which the trip operates).

- `route_id` – reference to the route this trip belongs to (foreign key to `routes.txt`)
- `service_id` – reference to the service calendar defining when this trip runs (foreign key to `calendar.txt` / `calendar_dates.txt`)
- `trip_id` – unique identifier of the trip
- `trip_headsign` – text shown on the vehicle's destination display (e.g., Sędziszów, Wrocław Główny)
- `direction_id` – binary direction indicator (0 or 1) distinguishing inbound from outbound trips on the same route
- `block_id` – identifies the physical vehicle block, allowing the linking of consecutive trips served by the same vehicle

3.5 stop_times.txt – Stop time entries (48 727 entries)

The largest file in the dataset. Contains the arrival and departure time of every trip at every stop it serves, defining the exact sequence and timing of stops within each trip. This is the primary file for constructing the transit graph.

- `trip_id` – reference to the trip (foreign key to `trips.txt`)
- `arrival_time` – scheduled arrival time at the stop, formatted as HH:MM:SS. Note: values may exceed 23:59:59 for trips running past midnight (e.g., 25:10:00 means 01:10 the next day)
- `departure_time` – scheduled departure time from the stop, formatted as HH:MM:SS (same overflow convention as `arrival_time`)
- `stop_id` – reference to the stop served (foreign key to `stops.txt`)
- `stop_sequence` – non-negative integer defining the order of stops within the trip (starts from 0)
- `stop_headsign` – optional override of the trip's headsign at this stop
- `pickup_type` – indicates whether passengers can board at this stop: 0 = regular pickup, 1 = no pickup available
- `shape_dist_traveled` – distance traveled along the route shape up to this stop (may be empty)

3.6 calendar.txt – Service calendars (3 260 entries)

Defines regular weekly service patterns. Each entry specifies which days of the week a given `service_id` is active, along with the validity period (start and end dates in YYYYMMDD format).

- `service_id` – unique identifier of the service pattern
- `monday ... sunday` – binary flags (0 or 1) indicating whether the service runs on that day of the week
- `start_date` – first day of the service validity period, formatted as YYYYMMDD
- `end_date` – last day of the service validity period, formatted as YYYYMMDD

3.7 calendar_dates.txt – Service exceptions (5 549 entries)

Defines date-specific additions or removals of service. This file overrides the weekly patterns from `calendar.txt` on specific dates (e.g., public holidays, special events).

- `service_id` – reference to the affected service pattern
- `date` – the specific date of the exception, formatted as YYYYMMDD
- `exception_type` – 1 = service *added* on this date, 2 = service *removed* on this date

3.8 feed_info.txt – Feed metadata

Contains metadata about the GTFS feed itself, including the publisher and the validity period of the dataset.

- `feed_publisher_name` – name of the feed publisher (`kiedyPrzyjedzie.pl`)
- `feed_publisher_url` – publisher's website URL
- `feed_lang` – language of the feed (`pl`)
- `feed_start_date` – first day for which the feed data is valid (20260303)
- `feed_end_date` – last day for which the feed data is valid (20261212)

3.9 Constructing the transit graph

In order to use the provided GTFS data for pathfinding, the student must construct a directed, time-dependent graph. The following guidelines describe how the files relate to one another and how edges should be formed:

1. **Nodes.** Each node in the graph represents a stop (or platform) as defined in `stops.txt`. The geographic coordinates (`stop_lat`, `stop_lon`) can be used to compute heuristic distance estimates for the A* algorithm (e.g., using the Haversine formula or Euclidean approximation on projected coordinates).
2. **Trip edges (direct connections).** A directed edge exists between two consecutive stops within the same trip. For each trip in `trips.txt`, the file `stop_times.txt` provides the ordered sequence of stops (sorted by `stop_sequence`). For consecutive entries $(i, i + 1)$ in this sequence, create an edge from `stop_idi` to `stop_idi+1` with:
 - departure time = `departure_timei`
 - arrival time = `arrival_timei+1`
 - travel time = `arrival_timei+1` – `departure_timei`
 - line name = `route_short_name` (or `route_long_name` if the short name is empty), obtained by joining through `trips.txt` → `routes.txt`
3. **Transfer edges.** When a passenger changes from one line to another at the same station, a transfer occurs. Transfers should be modeled by connecting stops that share the same `parent_station` (or have the same `stop_id` if no parent-child hierarchy exists). A transfer incurs a waiting time (the difference between the arrival of the first trip and the departure of the next trip at the same station) and increments the transfer counter by one.
4. **Service day filtering.** Not all trips run every day. To determine whether a trip with a given `service_id` is active on a particular date, combine the weekly schedule from `calendar.txt` with the exceptions in `calendar_dates.txt`. The student's application should accept a starting date (or use a default date within the validity period) and only consider trips that are active on that date.

4 Tasks

1. With the use of the provided GTFS dataset from Koleje Dolnośląskie, implement an algorithm searching for shortest paths between given stops A and B. As a cost function use (depending on the user's decision) travel time from A to B or the number of transfers needed.

Application should accept input in form of 4 variables:

- (a) the starting stop A
- (b) the ending stop B
- (c) optimization criterion: value t denotes travel time minimization, value p means minimization of the number of transfers
- (d) start time

The solution should print out to the standard output in successive lines detailed path information including starting stop, ending stop, used line's name, starting time, ending time, and on the standard error output the value of minimized criterion and time required for calculation of the shortest path.

Scoring:

- (a) shortest path search from A to B using Dijkstra's algorithm, based on travel time criterion (10 points).
 - (b) shortest path search from A to B using A* algorithm, based on travel time criterion (25 points).
 - (c) shortest path search from A to B using A* algorithm, based on the number of transfers criterion (25 points).
 - (d) modifications of A* algorithm from (b) or (c), allowing the reduction of computational time or improving quality of found solution (10 points)
2. Using the provided GTFS dataset, implement the shortest path from a given stop A that visits all the stops given in a list $L = A_2, \dots, A_n$, and returns back to A. As a cost function use (depending on the user's decision) travel time from A to B or the number of transfers needed.

The application should accept input consisting of 4 variables:

- (a) starting stop A
- (b) list of stops to visit separated by a semicolon L
- (c) optimization criterion: value t denotes travel time minimization, value p means minimization of the number of transfers
- (d) starting time

The solution should print out to the standard output, in consecutive lines, the detailed path information including starting stop, ending stop, used line's name, starting time, ending time, and on the standard error output the value of the minimized criterion and time required for calculation of the shortest path.

Scoring:

- (a) an algorithm searching for the shortest path between the nodes based on Tabu Search without bounding the size of T (10 points)
- (b) modification of the algorithm (a) with a variable T size, based on the length of L, in order to minimize the cost function (5 points)
- (c) modification of the algorithm (a) by adding Aspiration criterion, in order to minimize the cost function (5 points)
- (d) modification of the algorithm (a) by adding a neighbor sampling strategy to minimize the cost function and reduce the computation time. (10 points)

For each of the tasks prepare a report containing the theoretical background of the used algorithms, an example of using the method, the description of added modifications to the base algorithms, and additional materials used in order to fulfill the task as well as a short description of the libraries used in the implementation. In summary, describe the problems encountered during the implementation. Reports have to be sent to the tutor at least 24h before the tasks' submission.

5 Bibliography

- GTFS Schedule Reference – official specification of the General Transit Feed Specification
- An interactive animation describing A*
- Forbot's presentation on A*
- Blog post concerning A* and its steps
- A video describing A*
- The original paper on A*
- E. Taillard – a survey on Tabu search
- Knox – Tabu search for a symmetric TSP
- Lai, Demirag, Leung – Tabu search for a vehicle routing
- Silvestrin, Ritt – Tabu search for a vehicle routing
- Soto et al. – Tabu search for a vehicle routing
- Lin, Bian, Liu – a Tabu search – simulated annealing hybrid with a dynamic neighborhood selection for TSP